

Solution 1

Solution 1 (a)

Let $x = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$. To find the unit vector u , we must divide x by its L_2 -norm, $\|x\|_2 = \sqrt{x_1^2 + x_2^2}$.

$$\begin{aligned}\|x\|_2 &= \sqrt{3^2 + 4^2} \\ &= \sqrt{9 + 16} \\ &= \sqrt{25} \\ &= 5\end{aligned}$$

Now, we compute $u = \frac{x}{\|x\|_2}$.

$\therefore$ the unit vector u is the following:

$$u = \frac{1}{5} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix}$$

Solution 1

Solution 1 (b)

Let $x = \begin{bmatrix} -4 \\ 12 \\ 3 \end{bmatrix}$. First, we find the L_2 -norm of x , $\|x\|_2 = \sqrt{x_1^2 + x_2^2 + x_3^2}$.

$$\begin{aligned}\|x\|_2 &= \sqrt{(-4)^2 + 12^2 + 3^2} \\ &= \sqrt{16 + 144 + 9} \\ &= \sqrt{169} \\ &= 13\end{aligned}$$

The unit vector u is $u = \frac{x}{\|x\|_2}$.

$\therefore$ the unit vector u is the following:

$$u = \frac{1}{13} \begin{bmatrix} -4 \\ 12 \\ 3 \end{bmatrix} = \begin{bmatrix} \frac{-4}{13} \\ \frac{12}{13} \\ \frac{3}{13} \end{bmatrix}$$

Solution 1

Solution 1 (c)

Let $x = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$ such that $|x| = d$. This vector consists of d entries, each equal to 1. We find its L_2 -norm:

$$\begin{aligned}\|x\|_2 &= \sqrt{\sum_{i=1}^d 1^2} \\ &= \sqrt{1 + 1 + \cdots + 1} \quad (d \text{ times}) \\ &= \sqrt{d}\end{aligned}$$

The unit vector u is $u = \frac{x}{\|x\|_2}$.

$\therefore$ the unit vector u is the following:

$$u = \frac{1}{\sqrt{d}} \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{d}} \\ \frac{1}{\sqrt{d}} \\ \vdots \\ \frac{1}{\sqrt{d}} \end{bmatrix}$$

Solution 2

Solution 2 (a)

Given point $x = \begin{bmatrix} -2 \\ 4 \end{bmatrix}$ and direction $u = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$. The projection of vector x onto vector u is given by the formula:

$$\text{proj}_u x = \frac{x \cdot u}{\|u\|^2} u$$

First, calculate the dot product $x \cdot u$:

$$\begin{aligned} x \cdot u &= (-2)(1) + (4)(2) \\ &= -2 + 8 \\ &= 6 \end{aligned}$$

Next, calculate the squared L_2 -norm of u , $\|u\|^2$:

$$\begin{aligned} \|u\|^2 &= 1^2 + 2^2 \\ &= 1 + 4 \\ &= 5 \end{aligned}$$

Now, substitute these values into the projection formula:

$$\text{proj}_u x = \frac{6}{5} \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$\therefore$ the projection of point $x \in \mathbb{R}^2$ onto the direction of u is:

$$\text{proj}_u x = \begin{bmatrix} \frac{6}{5} \\ \frac{12}{5} \end{bmatrix} = \begin{bmatrix} 1.2 \\ 2.4 \end{bmatrix}$$

Solution 2

Solution 2 (b)**Plot**

The projection is calculated to be:

$$\text{proj}_u x = \begin{bmatrix} 1.2 \\ 2.4 \end{bmatrix}$$

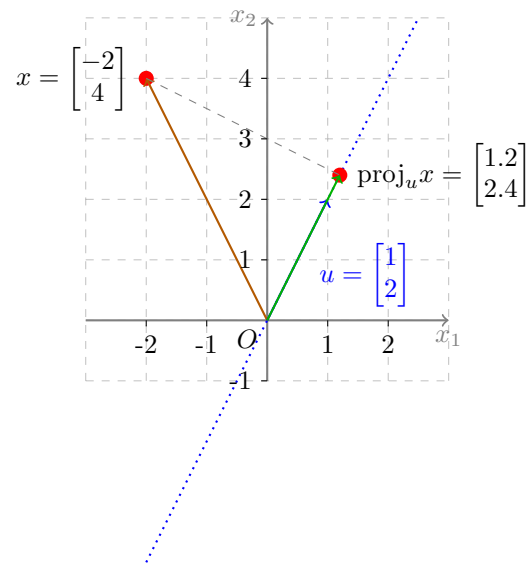


Figure 1: Projection of point x onto direction u .

Solution 3

Solution 3 (a)**Maximized Projection**

Let,

$$x = \begin{bmatrix} -4 \\ 3 \end{bmatrix}$$

We can define the dot product as:

$$x \cdot u = |x||u|\cos(\theta) \rightarrow \frac{x \cdot u}{|u|} = |x|\cos(\theta)$$

The projection of x onto a unit vector u is:

$$\text{proj}_u x = \frac{x \cdot u}{\|u\|^2} u$$

Hence, the project of x onto u is maximized when u is in the direction of x or (i.e., $\theta = 0$). It follows that:

$$\begin{aligned} \frac{x_1 u_1 + x_2 u_2}{\sqrt{u_1^2 + u_2^2}} &= \sqrt{x_1^2 + x_2^2} \cos(\theta) \\ \frac{-4u_1 + 3u_2}{\sqrt{u_1^2 + u_2^2}} &= \sqrt{(-4)^2 + 3^2} \cos(0) \\ \frac{-4u_1 + 3u_2}{\sqrt{u_1^2 + u_2^2}} &= 5 \\ \frac{-\frac{4u_1}{5} + \frac{3}{5}u_2}{\sqrt{u_1^2 + u_2^2}} &= 1 \end{aligned}$$

$\therefore$ the unit vector u is the following:

$$u = \begin{bmatrix} -\frac{4}{5} \\ \frac{3}{5} \end{bmatrix}$$

Solution 3

Solution 3 (b)**Minimized Projection**

Let,

$$x = \begin{bmatrix} -4 \\ 3 \end{bmatrix}$$

We can define the dot product as:

$$x \cdot u = \|x\| \|u\| \cos(\theta) \rightarrow \frac{x \cdot u}{\|u\|} = \|x\| \cos(\theta)$$

The projection of x onto a unit vector u is:

$$\text{proj}_u x = \frac{x \cdot u}{\|u\|^2} u.$$

Hence, the projection of x onto u is minimized when u is opposite to x (i.e., $\theta = \pi$). It follows that:

$$\begin{aligned} \frac{x_1 u_1 + x_2 u_2}{\sqrt{u_1^2 + u_2^2}} &= \sqrt{x_1^2 + x_2^2} \cos(\theta) \\ \frac{-4u_1 + 3u_2}{\sqrt{u_1^2 + u_2^2}} &= \sqrt{(-4)^2 + 3^2} \cos(\pi) \\ \frac{-4u_1 + 3u_2}{\sqrt{u_1^2 + u_2^2}} &= 5(-1) = -5 \\ \frac{-\frac{4}{5}u_1 + \frac{3}{5}u_2}{\sqrt{u_1^2 + u_2^2}} &= -1 \end{aligned}$$

$\therefore$ the unit vector u is the following:

$$u = \begin{bmatrix} \frac{4}{5} \\ -\frac{3}{5} \end{bmatrix}$$

Solution 3

Solution 3 (c)**Zero Projection**

Let,

$$x = \begin{bmatrix} -4 \\ 3 \end{bmatrix}$$

We can define the dot product as:

$$x \cdot u = \|x\| \|u\| \cos(\theta).$$

The projection of x onto u is zero when $x \cdot u = 0$ (i.e., $\theta \in \{\frac{\pi}{2}, \frac{3\pi}{2}\}$). For $u = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$,

$$x \cdot u = -4u_1 + 3u_2 = 0 \implies 3u_2 = 4u_1 \implies u_2 = \frac{4}{3}u_1.$$

A direction orthogonal to x is $v = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ since $x \cdot v = -12 + 12 = 0$, and $\|v\| = \sqrt{3^2 + 4^2} = 5$. Normalizing yields the unit directions:

$$u^{(1)} = \frac{1}{5} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} \frac{3}{5} \\ \frac{4}{5} \end{bmatrix}, \quad u^{(2)} = -u^{(1)} = \begin{bmatrix} -\frac{3}{5} \\ -\frac{4}{5} \end{bmatrix}.$$

$\therefore$ the unit vector(s) u are the following:

$$u^{(1)} = \begin{bmatrix} \frac{3}{5} \\ \frac{4}{5} \end{bmatrix} \quad \text{and} \quad u^{(2)} = \begin{bmatrix} -\frac{3}{5} \\ -\frac{4}{5} \end{bmatrix}$$

Solution 4

Solution 4 (a)

Let Σ and direction x ,

$$\Sigma = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 9 & 0 \\ 0 & 0 & 4 \end{bmatrix} \quad \text{and} \quad x = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

The variance in direction x is $x^\top \Sigma x$.

$$\begin{aligned} \text{Var}(x) &= x^\top \Sigma x \\ &= \begin{bmatrix} 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 & 0 \\ 2 & 9 & 0 \\ 0 & 0 & 4 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} (1 \cdot 0) + (2 \cdot 1) + (0 \cdot 0) \\ (2 \cdot 0) + (9 \cdot 1) + (0 \cdot 0) \\ (0 \cdot 0) + (0 \cdot 1) + (4 \cdot 0) \end{bmatrix} \\ &= \begin{bmatrix} 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 9 \\ 0 \end{bmatrix} \\ &= (2 \cdot 0) + (9 \cdot 1) + (0 \cdot 0) \\ &= 9 \end{aligned}$$

$\therefore$ the variance in the direction $\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$ is 9.

Solution 4

Solution 4 (b)

Let Σ and unit direction x ,

$$\Sigma = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 9 & 0 \\ 0 & 0 & 4 \end{bmatrix} \quad \text{and} \quad x = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\begin{aligned} \text{Var}(x) &= x^\top \Sigma x \\ &= \left(\frac{1}{\sqrt{3}} [1 \quad 1 \quad 1] \right) \begin{bmatrix} 1 & 2 & 0 \\ 2 & 9 & 0 \\ 0 & 0 & 4 \end{bmatrix} \left(\frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right) \\ &= \left(\frac{1}{\sqrt{3}} \cdot \frac{1}{\sqrt{3}} \right) \begin{bmatrix} (1 \cdot 1) + (2 \cdot 1) + (0 \cdot 1) \\ (2 \cdot 1) + (9 \cdot 1) + (0 \cdot 1) \\ (0 \cdot 1) + (0 \cdot 1) + (4 \cdot 1) \end{bmatrix} \\ &= \frac{1}{3} [1 \quad 1 \quad 1] \begin{bmatrix} 3 \\ 11 \\ 4 \end{bmatrix} \\ &= \frac{1}{3} (3 + 11 + 4) \\ &= \frac{18}{3} \\ &= 6 \end{aligned}$$

$\therefore$ the variance in the direction $\frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$ is 6.

Solution 4

Solution 4 (c)

Let Σ be as above and $u = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} \in \mathbb{R}^3$.

$$\begin{aligned}
 u^\top \Sigma u &= \begin{bmatrix} u_1 & u_2 & u_3 \end{bmatrix} \begin{bmatrix} 1 & 2 & 0 \\ 2 & 9 & 0 \\ 0 & 0 & 4 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} \\
 &= \begin{bmatrix} u_1 & u_2 & u_3 \end{bmatrix} \begin{bmatrix} u_1 + 2u_2 \\ 2u_1 + 9u_2 \\ 4u_3 \end{bmatrix} \\
 &= u_1(u_1 + 2u_2) + u_2(2u_1 + 9u_2) + u_3(4u_3) \\
 &= u_1^2 + 4u_1u_2 + 9u_2^2 + 4u_3^2.
 \end{aligned}$$

Hence, we the form $au_1^2 + bu_2^2 + cu_3^2 + du_1u_2$, and a,b,c,d are:

$$a = 1, \quad b = 9, \quad c = 4, \quad d = 4$$

$\therefore$ the variance function is:

$$u^\top \Sigma u = u_1^2 + 4u_1u_2 + 9u_2^2 + 4u_3^2$$

Solution 5

Solution 5

Let Σ ,

$$\Sigma = \begin{bmatrix} 5 & 0 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 & 0 \\ 0 & 0 & 20 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 4 \end{bmatrix}$$

For any unit direction $u = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \end{bmatrix} \in \mathbb{R}^5$ with $\|u\|_2 = 1$, the variance in direction u is

$$\begin{aligned} \text{Var}(u) &= u^\top \Sigma u \\ &= \begin{bmatrix} u_1 & u_2 & u_3 & u_4 & u_5 \end{bmatrix} \begin{bmatrix} 5 & 0 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 & 0 \\ 0 & 0 & 20 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 4 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \end{bmatrix} \\ &= \begin{bmatrix} u_1 & u_2 & u_3 & u_4 & u_5 \end{bmatrix} \begin{bmatrix} (5u_1 + 0u_2 + 0u_3 + 0u_4 + 0u_5) \\ (0u_1 + 10u_2 + 0u_3 + 0u_4 + 0u_5) \\ (0u_1 + 0u_2 + 20u_3 + 0u_4 + 0u_5) \\ (0u_1 + 0u_2 + 0u_3 + 1u_4 + 0u_5) \\ (0u_1 + 0u_2 + 0u_3 + 0u_4 + 4u_5) \end{bmatrix} \\ &= \begin{bmatrix} u_1 & u_2 & u_3 & u_4 & u_5 \end{bmatrix} \begin{bmatrix} 5u_1 \\ 10u_2 \\ 20u_3 \\ 1u_4 \\ 4u_5 \end{bmatrix} \\ &= u_1(5u_1) + u_2(10u_2) + u_3(20u_3) + u_4(1u_4) + u_5(4u_5) \\ &= 5u_1^2 + 10u_2^2 + 20u_3^2 + u_4^2 + 4u_5^2 \end{aligned}$$

$\therefore$ the direction of maximum variance is:

$$u = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Solution 6

Solution 6 (a)**Mean and Variance of $X \cdot u$**

Let,

$$\mu = \mathbb{E}[X] = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \text{ and } \Sigma = \text{cov}(X) = \begin{bmatrix} 5 & -3 & 0 \\ -3 & 5 & 0 \\ 0 & 0 & 4 \end{bmatrix} \text{ and } u = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\begin{aligned} \mathbb{E}[X \cdot u] &= u^\top \mu \\ &= \frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \\ &= \frac{1}{\sqrt{3}} (1 + 2 + 3) \\ &= \frac{6}{\sqrt{3}} \end{aligned}$$

$$\begin{aligned} \text{Var}(X \cdot u) &= u^\top \Sigma u \\ &= \left(\frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} \right) \begin{bmatrix} 5 & -3 & 0 \\ -3 & 5 & 0 \\ 0 & 0 & 4 \end{bmatrix} \left(\frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right) \\ &= \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \\ 4 \end{bmatrix} \\ &= \frac{1}{3} (2 + 2 + 4) \\ &= \frac{8}{3} \end{aligned}$$

$$\therefore \mathbb{E}[X \cdot u] = \frac{6}{\sqrt{3}} \text{ and } \text{Var}(X \cdot u) = \frac{8}{3}.$$

Solution 6**Solution 6 (b)**

Let

$$\Sigma = \begin{bmatrix} 5 & -3 & 0 \\ -3 & 5 & 0 \\ 0 & 0 & 4 \end{bmatrix}, \quad v_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad v_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix},$$

$$v_4 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \quad v_5 = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \quad v_6 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}$$

Check each candidate via $\Sigma v = \lambda v$:

$$\Sigma v_1 = \begin{bmatrix} 5 \\ -3 \\ 0 \end{bmatrix} \neq \lambda \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \text{ for any } \lambda \quad \Rightarrow \quad v_1 \text{ is not an eigenvector.}$$

$$\Sigma v_2 = \begin{bmatrix} -3 \\ 5 \\ 0 \end{bmatrix} \neq \lambda \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ for any } \lambda \quad \Rightarrow \quad v_2 \text{ is not an eigenvector.}$$

$$\Sigma v_3 = \begin{bmatrix} 0 \\ 0 \\ 4 \end{bmatrix} = 4 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad \Rightarrow \quad v_3 \text{ is an eigenvector with } \lambda = 4.$$

$$\Sigma v_4 = \frac{1}{\sqrt{2}} \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix} = 2 \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \right) \quad \Rightarrow \quad v_4 \text{ is an eigenvector with } \lambda = 2.$$

$$\Sigma v_5 = \frac{1}{\sqrt{2}} \begin{bmatrix} -3 \\ 5 \\ 4 \end{bmatrix} \neq \lambda \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \right) \text{ for any } \lambda \quad \Rightarrow \quad v_5 \text{ is not an eigenvector.}$$

$$\Sigma v_6 = \frac{1}{\sqrt{2}} \begin{bmatrix} 8 \\ -8 \\ 0 \end{bmatrix} = 8 \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} \right) \quad \Rightarrow \quad v_6 \text{ is an eigenvector with } \lambda = 8.$$

 $\therefore$ From the provided list, the eigenvectors are:

$$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \text{ with eigenvalue 2, } \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} \text{ with eigenvalue 8, and } \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \text{ with eigenvalue 4}$$

Solution 6

Solution 6 (c)**Eigenvalues for the eigenvectors from (b)**

From the computations in (b),

$$\lambda \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \right) = 2, \quad \text{eigenvalue: } 2$$

$$\lambda \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} \right) = 8, \quad \text{eigenvalue: } 8$$

$$\lambda \left(\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right) = 4, \quad \text{eigenvalue: } 4$$

Solution 6

Solution 6 (d)**PCA: 2D projection directions**

Eigenvalues (descending) are 8, 4, 2. Thus the top two principal directions are the eigenvectors for 8 and 4:

$$v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

$\therefore$ PCA projects onto $\left\{ \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}$

Solution 6

Solution 6 (e)**2D PCA projection**

Let,

$$x = \begin{bmatrix} 4 \\ 0 \\ 2 \end{bmatrix} \quad \mu = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} \quad v_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

1. Center x by μ

$$x - \mu = \begin{bmatrix} 4 - 1 \\ 0 - 2 \\ 2 - 3 \end{bmatrix} = \begin{bmatrix} 3 \\ -2 \\ -1 \end{bmatrix}$$

2. Find y_1

$$\begin{aligned} y_1 &= v_1^\top (x - \mu) \\ &= \frac{1}{\sqrt{2}} [1 \quad -1 \quad 0] \begin{bmatrix} 3 \\ -2 \\ -1 \end{bmatrix} \\ &= \frac{1}{\sqrt{2}} (3 + 2) \\ &= \frac{5}{\sqrt{2}} \end{aligned}$$

3. Find y_2

$$\begin{aligned} y_2 &= v_2^\top (x - \mu) \\ &= [0 \quad 0 \quad 1] \begin{bmatrix} 3 \\ -2 \\ -1 \end{bmatrix} \\ &= -1 \end{aligned}$$

$\therefore$ The 2D PCA coordinates are $\begin{bmatrix} \frac{5}{\sqrt{2}} \\ -1 \end{bmatrix}$

Solution 6

Solution 6 (f)**Reconstruction from 2D PCA**

Let,

$$\mu = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad y_1 = \frac{5}{\sqrt{2}} \quad y_2 = -1 \quad v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} \quad v_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\begin{aligned} x &= \mu + y_1 v_1 + y_2 v_2 \\ &= \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + \frac{5}{\sqrt{2}} \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} \right) + (-1) \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + \frac{5}{2} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ -1 \end{bmatrix} \\ &= \begin{bmatrix} 1 + \frac{5}{2} \\ 2 - \frac{5}{2} \\ 3 - 1 \end{bmatrix} \\ &= \begin{bmatrix} \frac{7}{2} \\ -\frac{1}{2} \\ 2 \end{bmatrix} \end{aligned}$$

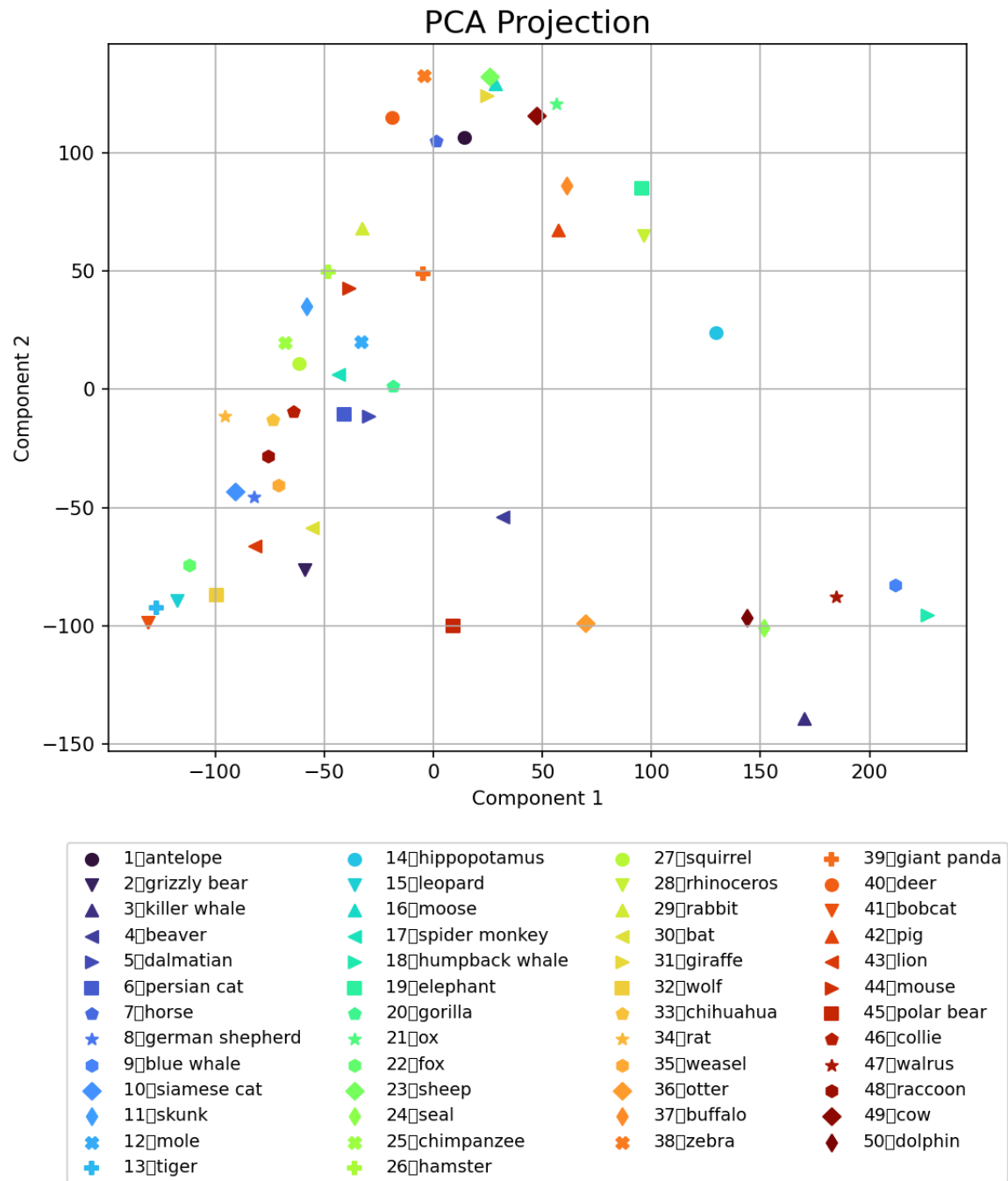
$\therefore$ The 3D reconstruction is:

$$x = \begin{bmatrix} \frac{7}{2} \\ -\frac{1}{2} \\ 2 \end{bmatrix}$$

Solution 7

Solution 7 (a)

PCA 2D Projection

Figure 2: PCA projection $\mathbb{R}^{85}$ to $\mathbb{R}^2$

Solution 7

Solution 7 (a)

PCA Code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.decomposition import PCA
4 from sklearn.manifold import TSNE
5 from scipy.spatial.distance import pdist, squareform
6 import re
7
8 # set plot figure size for better readability
9 plt.rcParams['figure.figsize'] = (6, 8)
10
11 def load_animal_names(filepath='classes.txt'):
12     with open(filepath, 'r') as f:
13         names = [line.strip() for line in f.readlines()]
14         names = [name.split('.')[0].replace('+', ' ') for name in names]
15     return(names)
16
17 def main():
18     # read predicate-matrix-continuous.txt
19     X = np.loadtxt('predicate-matrix-continuous.txt')
20     # read classes.txt
21     animal_names = load_animal_names('classes.txt')
22
23     # initialize list to hold embeddings and their titles
24     embeddings_to_plot = []
25     # pca model
26     pca = PCA(n_components=2, random_state=42)
27     Z_pca = pca.fit_transform(X)
28     embeddings_to_plot.append(('PCA Projection', Z_pca))
```

Solution 7

Solution 7 (b)

t-SNE Code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.decomposition import PCA
4 from sklearn.manifold import TSNE
5 from scipy.spatial.distance import pdist, squareform
6 import re
7
8 # set plot figure size for better readability
9 plt.rcParams['figure.figsize'] = (6, 8)
10
11 def load_animal_names(filepath='classes.txt'):
12     with open(filepath, 'r') as f:
13         names = [line.strip() for line in f.readlines()]
14         names = [name.split('.')[0].replace('+', ' ') for name in names]
15     return(names)
16
17 def main():
18     # read predicate-matrix-continuous.txt
19     X = np.loadtxt('predicate-matrix-continuous.txt')
20     # read classes.txt
21     animal_names = load_animal_names('classes.txt')
22
23     # initialize list to hold embeddings and their titles
24     embeddings_to_plot = []
25
26     # note: 49 ~ 50 its just python indexing
27     perplexities = [5, 10, 25, 49]
28     for p in perplexities:
29         print(f"Running t-SNE with perplexity={p}...")
30         tsne = TSNE(n_components=2, perplexity=p, random_state=42,
31                     init='pca', learning_rate='auto')
32         Z_tsne = tsne.fit_transform(X)
33         embeddings_to_plot.append((f't-SNE (Perplexity={p})', Z_tsne))
```

Solution 7

Solution 7 (c)**Evaluate Models**

Plot	Description	Average Distortion
pca_projection.png	PCA Projection	1.8313
t_sne_perplexity5.png	t-SNE (Perplexity=5)	1.8602
t_sne_perplexity10.png	t-SNE (Perplexity=10)	1.7101
t_sne_perplexity25.png	t-SNE (Perplexity=25)	1.5683
t_sne_perplexity49.png	t-SNE (Perplexity=49)	1.7370

Table 1: The embedding with the lowest average distortion is t-SNE (Perplexity=25) with a minimum distortion of 1.5683.

Solution 7

Solution 7 (c)

Distortion and Plot Code

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.decomposition import PCA
4 from sklearn.manifold import TSNE
5 from scipy.spatial.distance import pdist, squareform
6 import re
7
8 # set plot figure size for better readability
9 plt.rcParams['figure.figsize'] = (6, 8)
10
11 def load_animal_names(filepath='classes.txt'):
12     with open(filepath, 'r') as f:
13         names = [line.strip() for line in f.readlines()]
14         names = [name.split('.')[0].replace('+', ' ') for name in names]
15     return(names)
16
17 def compute_distortion(X_original, Z_embedded):
18     # calc pairwise distances for original space (n*(n-1)/2 vector)
19     D = pdist(X_original, metric='euclidean')
20
21     # calc pairwise distances for embedded space (n*(n-1)/2 vector)
22     D_hat = pdist(Z_embedded, metric='euclidean')
23
24     # add a small epsilon to avoid division by zero
25     epsilon = 1e-12
26     D[D == 0] = epsilon
27     D_hat[D_hat == 0] = epsilon
28
29     # calc scaling factor c (sum of all pairwise distances)
30     c = np.sum(D) / np.sum(D_hat)
31
32     # calc scaled embedded distances
33     scaled_D_hat = c * D_hat
34
35     # calculate distortion for each pair (i, j) with i < j
36     Delta_values = np.maximum(D / scaled_D_hat, scaled_D_hat / D)
37
38     return(np.mean(Delta_values))
39
40 def main():
41     # read predicate-matrix-continuous.txt
42     X = np.loadtxt('predicate-matrix-continuous.txt')
43     # read classes.txt
44     animal_names = load_animal_names('classes.txt')
45
46     # initialize list to hold embeddings and their titles
47     embeddings_to_plot = []
48
49     for title, Z in embeddings_to_plot:
50         # set plt configs
51         fig = plt.figure(figsize=(7.5, 9))
52         grid_shape = (3, 3)
53         ax_plot = plt.subplot2grid(grid_shape, (0, 0), rowspan=2, colspan=3)
54         ax_legend = plt.subplot2grid(grid_shape, (2, 0), rowspan=1, colspan=3)
55
56         # plot data
57         for j, name in enumerate(animal_names):
58             color = colors[j]
59             marker = markers[j % len(markers)]
60
61             ax_plot.scatter(Z[j, 0], Z[j, 1],
62                             color=color,
63                             marker=marker,
64                             label=name,

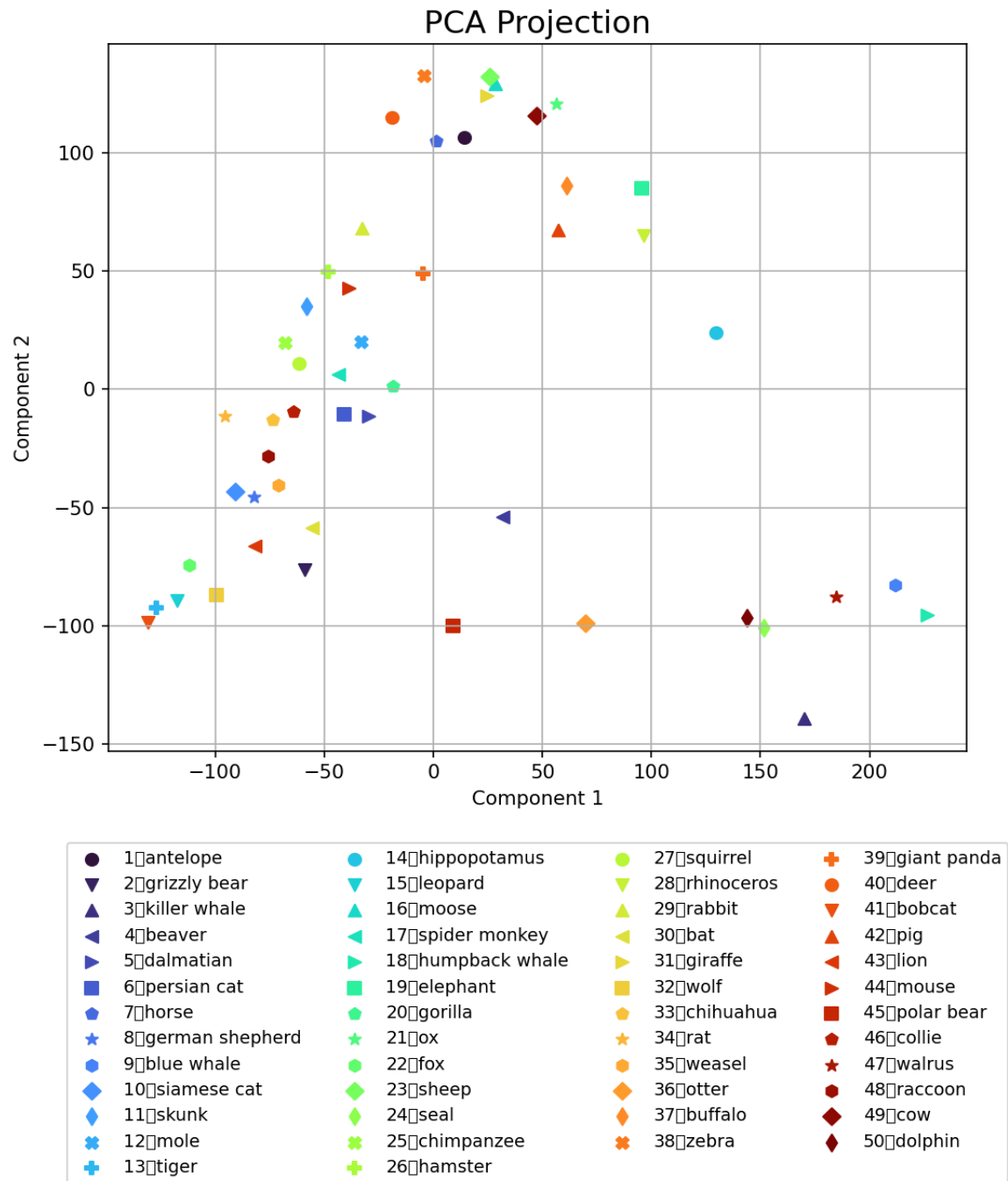
```

```
65         s=40) # s=40 is a reasonable marker size
66
67     ax_plot.set_title(title, fontsize=16)
68     ax_plot.set_xlabel('Component 1')
69     ax_plot.set_ylabel('Component 2')
70     ax_plot.grid(True)
71
72     # set legend settings
73     handles, labels = ax_plot.get_legend_handles_labels()
74     ax_legend.axis('off')
75     ax_legend.legend(handles, labels, loc='center', ncol=4, fontsize=9)
76
77     # save plot
78     # Use tight_layout to ensure no overlap
79     plt.tight_layout()
80     clean_title = title.lower()
81     clean_title = re.sub(r'[\W\s-]', '', clean_title)
82     clean_title = re.sub(r'[\s-]+', '_', clean_title).strip('_')
83     filename = f"{clean_title}.png"
84
85     plt.savefig(filename, dpi=150) # 150 dpi is good for print
86     plt.close(fig) # Close the figure to free memory
87
88     # calc distortion
89     distortions = {}
90
91     # store all embeddings in a dict for distortion calculation
92     all_embeddings_dict = {title: Z for title, Z in embeddings_to_plot}
93
94     for key, Z in all_embeddings_dict.items():
95         distortion = compute_distortion(X, Z)
96         distortions[key] = distortion
97         print(f"{key} average distortion: {distortion:.4f}")
98
99     best_embedding = min(distortions, key=distortions.get)
100     min_distortion = distortions[best_embedding]
101
102     print(f"lowest average distortion is: {best_embedding}")
103     print(f"minimum distortion: {min_distortion:.4f}")
```


Solution 7

Solution 7 (c)

PCA 2D Projection

Figure 3: PCA projection $\mathbb{R}^{85}$ to $\mathbb{R}^2$

Solution 7

Solution 7 (c)

t-SNE: Perplexity 5

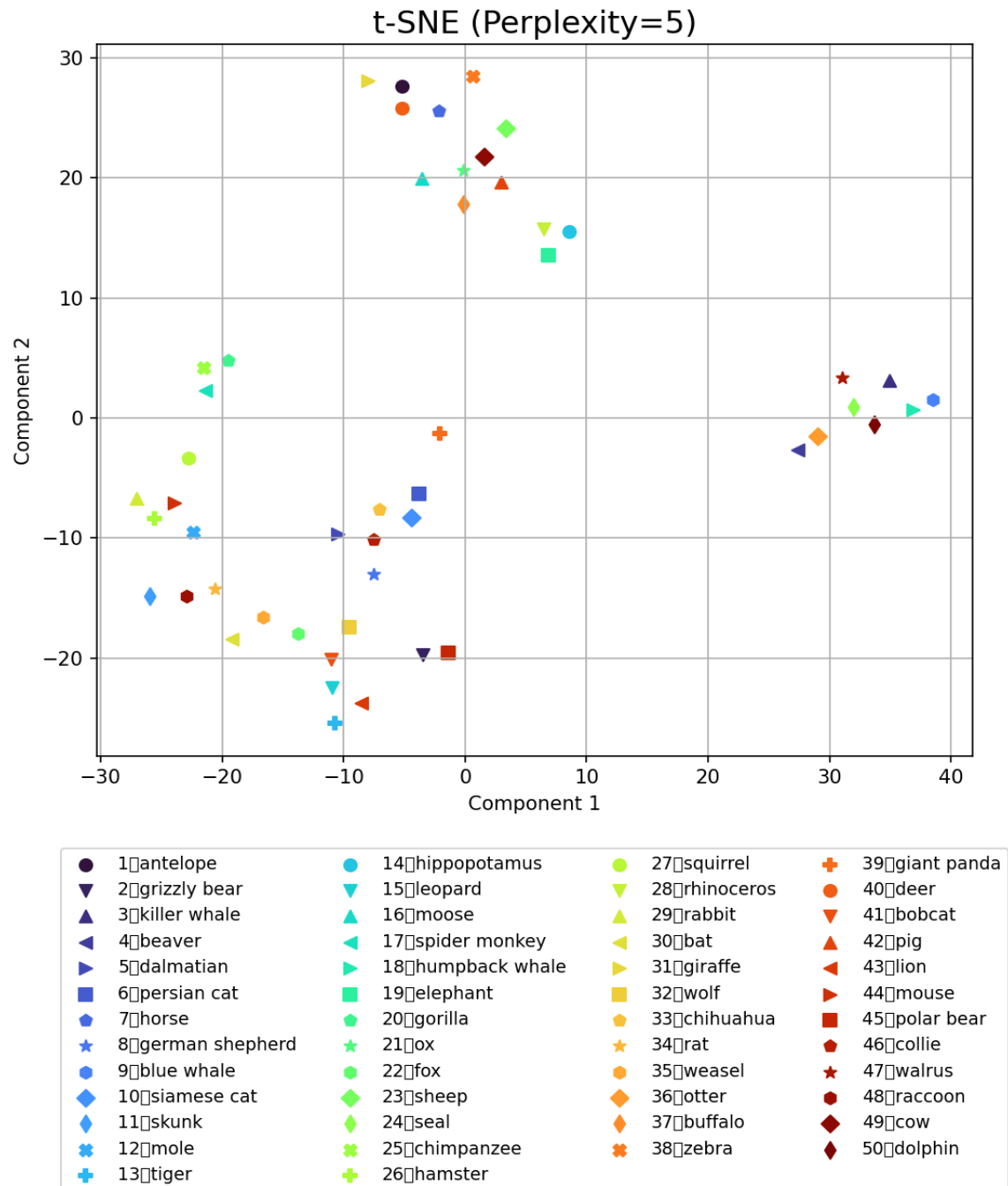


Figure 4: t-SNE plot with perplexity=5

Solution 7

Solution 7 (c)

t-SNE: Perplexity 10

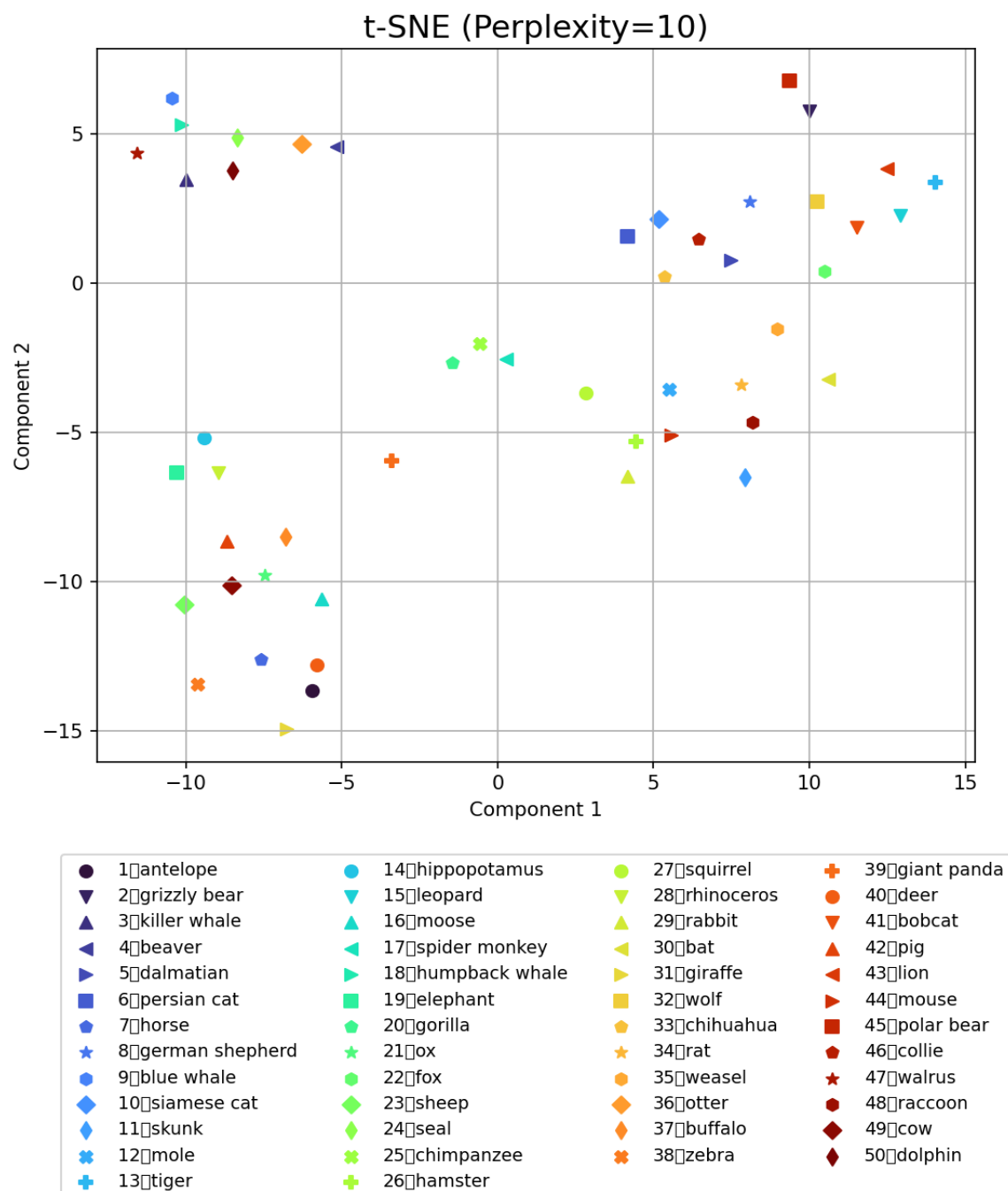


Figure 5: t-SNE plot with perplexity=10

Solution 7

Solution 7 (c)

t-SNE: Perplexity 25

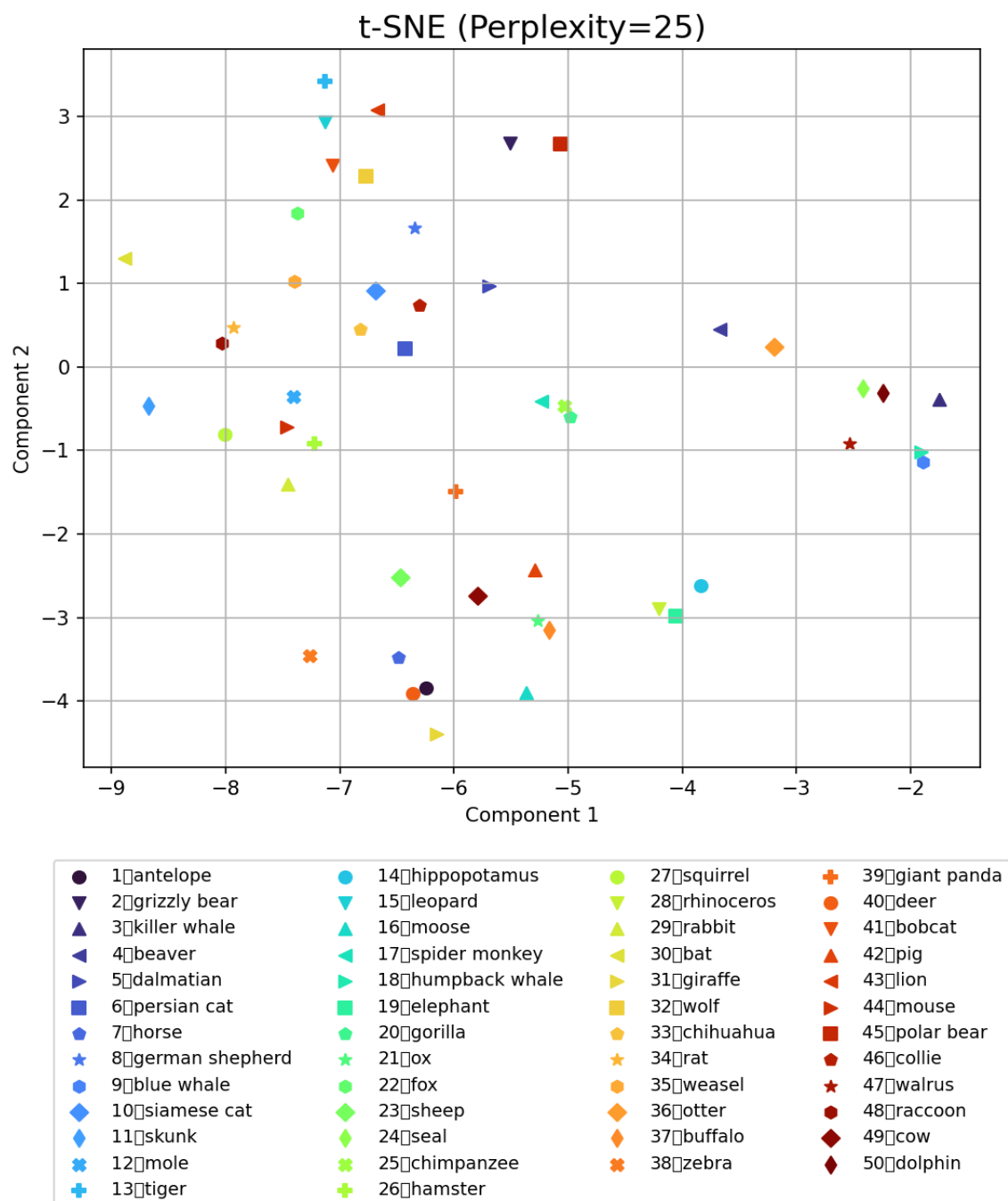


Figure 6: t-SNE plot with perplexity=25

Solution 7

Solution 7 (c)

t-SNE: Perplexity 49

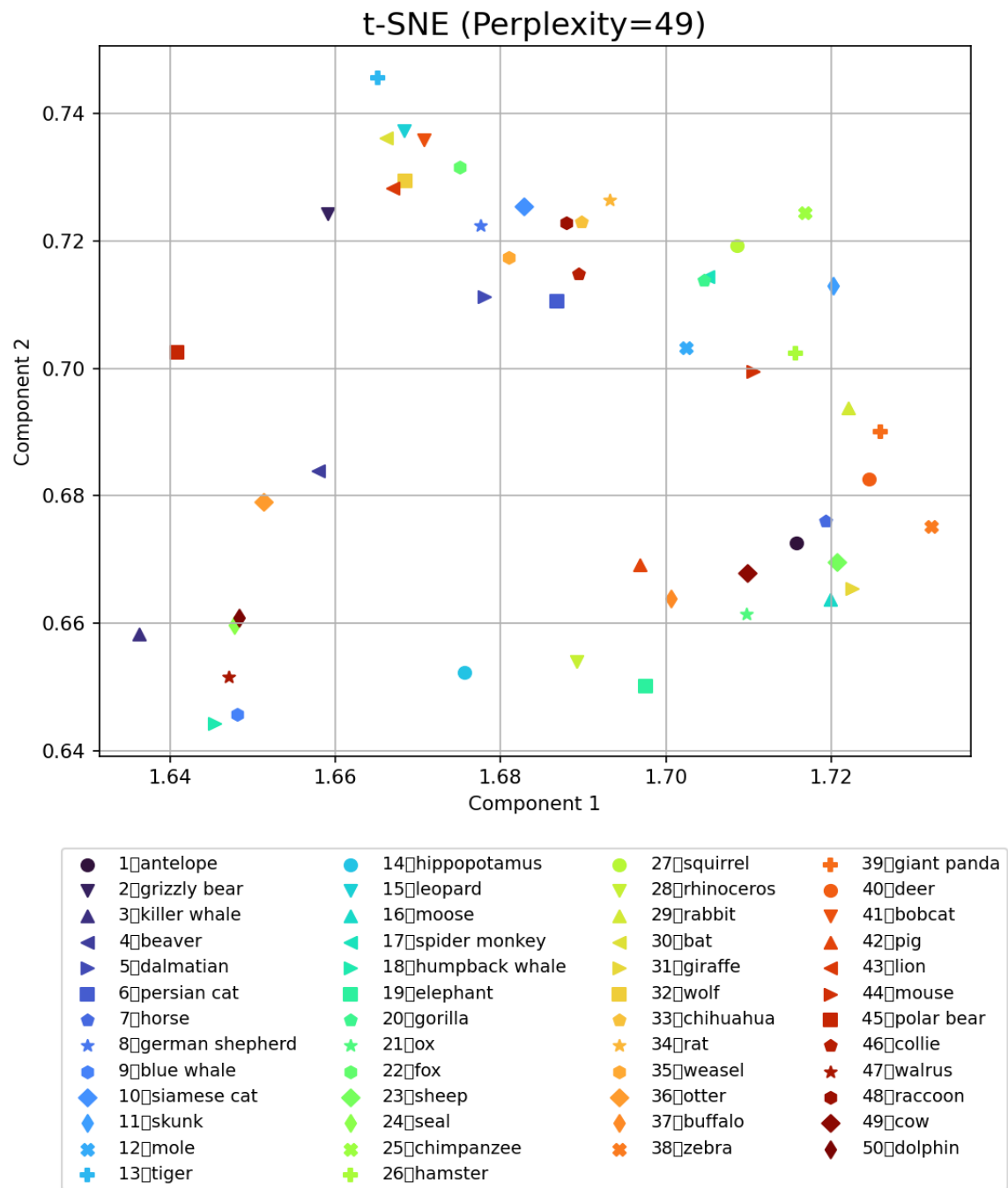


Figure 7: t-SNE plot with perplexity=49. Note: 49 and not 50 because of python indexing starts at 0

Solution 8

Best rank-2 approx to M

Let,

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

The SVD of M is given in the form $M = U\Sigma V^T = \sum_{i=1}^4 \sigma_i u_i v_i^T$. The best rank-2 approximation, M_2 , is found by taking the first two terms of the SVD sum, corresponding to the two largest singular values:

$$M_2 = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T$$

From the matrices U (columns u_i) and V^T (rows v_i^T), we identify the required components:

$$M = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}}_U \underbrace{\begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\Sigma_{4 \times 4}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}}_{V^T}$$

It follows that:

$$\sigma_1 = 4 \quad u_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad v_1^T = [1 \quad 0 \quad 0 \quad 0 \quad 0] \quad \sigma_2 = 3 \quad u_2 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad v_2^T = [0 \quad 1 \quad 0 \quad 0 \quad 0]$$

Now, calculate M_2 :

$$\begin{aligned} M_2 &= \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T \\ &= 4 \left(\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} [1 \quad 0 \quad 0 \quad 0 \quad 0] \right) + 3 \left(\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} [0 \quad 1 \quad 0 \quad 0 \quad 0] \right) \\ &= 4 \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} + 3 \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 4 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 \end{bmatrix} \end{aligned}$$

$\therefore$ The best rank-2 approximation to M is:

$$M_2 = \begin{bmatrix} 4 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 \end{bmatrix}$$

Solution 9

Solution 9 (a)

SVD

Python Code

```
1 import numpy as np
2
3 # set print sig figs
4 np.set_printoptions(precision=4, suppress=True)
5
6 # define matrix M
7 M = np.array([
8     [1, 2, 3],
9     [4, 5, 6]
10 ])
11
12 # calc "thin" SVD
13 U, s, Vt = np.linalg.svd(M, full_matrices=False)
14
15 # get the first singular value (sigma_1)
16 sigma_1 = s[0]
17
18 # get the first left singular vector (u_1)
19 u_1 = U[:, 0:1]
20
21 # get the first right singular vector (v_1^T)
22
23 v_1_T = Vt[0:1, :]
24
25 # calc rank-1 approximation
26 M_1 = sigma_1 * (u_1 @ v_1_T)
27
28 # print results
29 print(f"matrix M:\n{M}\n")
30 print("-" * 30)
31 print(f"sigma_1 (s[0]): {sigma_1:.4f}\n")
32 print(f"u_1 (U[:, 0]):\n{u_1}\n")
33 print(f"v_1^T (Vt[0, ]):\n{v_1_T}\n")
34 print("-" * 30)
35 print(f"best rank-1 approximation (M_1):\n{M_1}")
```

Solution 9

Solution 9 (a)**Compute M_1**

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

The best rank-1 approximation M_1 is:

$$M_1 = \sigma_1 u_1 v_1^T$$

From the Python output, we have:

$$\sigma_1 \approx 8.7950 \quad u_1 \approx \begin{bmatrix} -0.4042 \\ -0.9147 \end{bmatrix} \quad v_1^T \approx [-0.4287 \quad -0.5663 \quad -0.7038]$$

Now, calculate $M_1 = \sigma_1(u_1 v_1^T)$:

$$\begin{aligned} M_1 &\approx 8.7950 \left(\begin{bmatrix} -0.4042 \\ -0.9147 \end{bmatrix} [-0.4287 \quad -0.5663 \quad -0.7038] \right) \\ &\approx 8.7950 \begin{bmatrix} (-0.4042)(-0.4287) & (-0.4042)(-0.5663) & (-0.4042)(-0.7038) \\ (-0.9147)(-0.4287) & (-0.9147)(-0.5663) & (-0.9147)(-0.7038) \end{bmatrix} \\ &\approx 8.7950 \begin{bmatrix} 0.1733 & 0.2289 & 0.2845 \\ 0.3921 & 0.5179 & 0.6438 \end{bmatrix} \\ &\approx \begin{bmatrix} 1.5242 & 2.0131 & 2.5024 \\ 3.4485 & 4.5552 & 5.6621 \end{bmatrix} \end{aligned}$$

 $\therefore$ The best rank-1 approximation to M is:

$$M_1 \approx \begin{bmatrix} 1.524 & 2.013 & 2.502 \\ 3.449 & 4.555 & 5.662 \end{bmatrix}$$

Solution 9

Solution 9 (b)**Uniqueness**

Show the decomposition $M = uv^T$ for a rank-1 matrix is not unique.

Assume $M = uv^T$, where $u \in \mathbb{R}^p$ and $v \in \mathbb{R}^q$. Let c be any non-zero scalar (i.e., $c \in \mathbb{R}, c \neq 0$). Let a and b be vectors such that:

$$a = cu \in \mathbb{R}^p \text{ and } b = \left(\frac{1}{c}\right)v \in \mathbb{R}^q$$

If $c \neq 1$, then $a \neq u$ and $b \neq v$. It follows that (a, b) is a different pair of vectors from (u, v) . Next, compute the rank-1 matrix ab^T :

$$\begin{aligned} ab^T &= (cu) \left(\frac{1}{c}v\right)^T \\ &= (cu) \left(\frac{1}{c}v^T\right) \\ &= c \left(\frac{1}{c}\right) (uv^T) \\ &= 1 \cdot (uv^T) \\ &= uv^T \\ &= M \end{aligned}$$

Hence, $ab^T = uv^T = M$ and $a \neq u$ and $b \neq v$.

$\therefore$ The decomposition is not unique. For any non-zero scalar c , the pair $(cu, \frac{1}{c}v)$ will produce the same rank-1 matrix.

Code

Solution 7 Code

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.decomposition import PCA
4 from sklearn.manifold import TSNE
5 from scipy.spatial.distance import pdist, squareform
6 import re
7
8 # set plot figure size for better readability
9 plt.rcParams['figure.figsize'] = (6, 8)
10
11 def load_animal_names(filepath='classes.txt'):
12     with open(filepath, 'r') as f:
13         names = [line.strip() for line in f.readlines()]
14         names = [name.split('.')[0].replace('+', ' ') for name in names]
15     return(names)
16
17 def compute_distortion(X_original, Z_embedded):
18     # calc pairwise distances for original space (n*(n-1)/2 vector)
19     D = pdist(X_original, metric='euclidean')
20
21     # calc pairwise distances for embedded space (n*(n-1)/2 vector)
22     D_hat = pdist(Z_embedded, metric='euclidean')
23
24     # add a small epsilon to avoid division by zero
25     epsilon = 1e-12
26     D[D == 0] = epsilon
27     D_hat[D_hat == 0] = epsilon
28
29     # calc scaling factor c (sum of all pairwise distances)
30     c = np.sum(D) / np.sum(D_hat)
31
32     # calc scaled embedded distances
33     scaled_D_hat = c * D_hat
34
35     # calculate distortion for each pair (i, j) with i < j
36     Delta_values = np.maximum(D / scaled_D_hat, scaled_D_hat / D)
37
38     return(np.mean(Delta_values))
39
40 def main():
41     # read predicate-matrix-continuous.txt
42     X = np.loadtxt('predicate-matrix-continuous.txt')
43     # read classes.txt
44     animal_names = load_animal_names('classes.txt')
45
46     # initialize list to hold embeddings and their titles
47     embeddings_to_plot = []
48
49     # pca model
50     pca = PCA(n_components=2, random_state=42)
51     Z_pca = pca.fit_transform(X)
52     embeddings_to_plot.append(('PCA Projection', Z_pca))
53
54     # note: 49 ~ 50 its just python indexing
55     perplexities = [5, 10, 25, 49]
56     for p in perplexities:
57         print(f"Running t-SNE with perplexity={p}...")
58         tsne = TSNE(n_components=2, perplexity=p, random_state=42,
59                     init='pca', learning_rate='auto')
60         Z_tsne = tsne.fit_transform(X)
61         embeddings_to_plot.append(('t-SNE (Perplexity={p})', Z_tsne))
62
63     for title, Z in embeddings_to_plot:
64         # set plt configs
65         fig = plt.figure(figsize=(7.5, 9))
66         grid_shape = (3, 3)

```

```

67     ax_plot = plt.subplot2grid(grid_shape, (0, 0), rowspan=2, colspan=3)
68     ax_legend = plt.subplot2grid(grid_shape, (2, 0), rowspan=1, colspan=3)
69
70     # plot data
71     for j, name in enumerate(animal_names):
72         color = colors[j]
73         marker = markers[j % len(markers)]
74
75         ax_plot.scatter(Z[j, 0], Z[j, 1],
76                        color=color,
77                        marker=marker,
78                        label=name,
79                        s=40) # s=40 is a reasonable marker size
80
81     ax_plot.set_title(title, fontsize=16)
82     ax_plot.set_xlabel('Component 1')
83     ax_plot.set_ylabel('Component 2')
84     ax_plot.grid(True)
85
86     # set legend settings
87     handles, labels = ax_plot.get_legend_handles_labels()
88     ax_legend.axis('off')
89     ax_legend.legend(handles, labels, loc='center', ncol=4, fontsize=9)
90
91     # save plot
92     # Use tight_layout to ensure no overlap
93     plt.tight_layout()
94     clean_title = title.lower()
95     clean_title = re.sub(r'[\W\s-]', '', clean_title)
96     clean_title = re.sub(r'[\s-]+', '_', clean_title).strip('_')
97     filename = f"{clean_title}.png"
98
99     plt.savefig(filename, dpi=150) # 150 dpi is good for print
100    plt.close(fig) # Close the figure to free memory
101
102    # calc distortion
103    distortions = {}
104
105    # store all embeddings in a dict for distortion calculation
106    all_embeddings_dict = {title: Z for title, Z in embeddings_to_plot}
107
108    for key, Z in all_embeddings_dict.items():
109        distortion = compute_distortion(X, Z)
110        distortions[key] = distortion
111        print(f"{key} average distortion: {distortion:.4f}")
112
113    best_embedding = min(distortions, key=distortions.get)
114    min_distortion = distortions[best_embedding]
115
116    print(f"lowest average distortion is: {best_embedding}")
117    print(f"minimum distortion: {min_distortion:.4f}")

```

Code

Solution 9 Code

```
1 import numpy as np
2
3 # set print sig figs
4 np.set_printoptions(precision=4, suppress=True)
5
6 # define matrix M
7 M = np.array([
8     [1, 2, 3],
9     [4, 5, 6]
10 ])
11
12 # calc "thin" SVD
13 U, s, Vt = np.linalg.svd(M, full_matrices=False)
14
15 # get the first singular value (sigma_1)
16 sigma_1 = s[0]
17
18 # get the first left singular vector (u_1)
19 u_1 = U[:, 0:1]
20
21 # get the first right singular vector (v_1^T)
22
23 v_1_T = Vt[0:1, :]
24
25 # calc rank-1 approximation
26 M_1 = sigma_1 * (u_1 @ v_1_T)
27
28 # print results
29 print(f"matrix M:\n{M}\n")
30 print("-" * 30)
31 print(f"sigma_1 (s[0]): {sigma_1:.4f}\n")
32 print(f"u_1 (U[:, 0]):\n{u_1}\n")
33 print(f"v_1^T (Vt[0, :]):\n{v_1_T}\n")
34 print("-" * 30)
35 print(f"best rank-1 approximation (M_1):\n{M_1}")
```